



CÍRCULO POLICIAL DE FLORES

MANUAL DE USUARIO

Integración Auth0 (Angular + Spring Boot)

Autenticación y autorización mediante OAuth 2.0, OpenID Connect y JWT

Módulo / Alcance	Autenticación
Dirigido a	Equipo de desarrollo
Version	1.0
Estado	Aprobado
Fecha	29/07/2026



INFORMACIÓN DEL DOCUMENTO

Campo	Detalle
Título del documento	<i>Integración de Auth0 (Angular + Spring Boot)</i>
Módulo / Sistema	<i>Autenticación CIPOLFLO</i>
Tipo de manual	<i>Técnico</i>
Público objetivo	<i>Equipo de desarrollo</i>
Aplicación / versión del sistema	<i>CIPOLFLO — Sistema de Gestión</i>



Índice

Índice	3
1. Introducción.....	4
1.1 Objetivo del documento.....	4
1.2 Alcance y público destinatario.....	4
1.3 Requisitos previos.....	4
2. Configuración de Auth0	5
2.1 Configuración del tenant.....	5
2.2 Creación de la aplicación (SPA).....	5
2.3 Creación de Api.....	7
2.3 Variables de entorno	9
2.3 Bloque de comandos o valores técnicos	9
2.4 Validación de JWT.....	9
3. Preguntas frecuentes.....	10
4. Glosario	10
5. Anexos	10



1. Introducción

1.1 Objetivo del documento

Este manual describe cómo está configurada la integración con Auth0 en el backend de CIPOLFLO: la aplicación y API creadas en el tenant, las variables de entorno necesarias (AUTH0_ISSUER_URI, AUTH0_AUDIENCE), y cómo Spring Boot valida los JWT recibidos, incluyendo la validación de audiencia (AudienceValidator).

1.2 Alcance y público destinatario

Dirigido al equipo de desarrollo. Este documento cubre la configuración del sistema de autenticación, mas no el alta de usuarios ni la gestión de los accesos (Manual de usuario - Manejo de usuarios en AUTH0)

1.3 Requisitos previos

Antes de continuar, es necesario contar con lo siguiente:

- Acceso al tenant de Auth0 del proyecto
- Acceso al repositorio (cipolflo-server)

2. Configuración de Auth0

Auth0 es una plataforma de autenticación y autorización que permite implementar login seguro sin desarrollar un sistema de autenticación propio. Auth0 proporciona: gestión de usuarios, login seguro, manejo de JWT, soporte para OAuth2, OpenID Connect y autenticación multifactor (MFA).

2.1 Configuración del tenant

Configuración	Valor
Tenant Domain	dev-jkssh7xkp4k2krwe
Region	US
Resultado	dev-jkssh7xkp4k2krwe.us.auth0.com

2.2 Creación de la aplicación (SPA)

En el panel de Auth0: **Applications → Applications → Create Application**

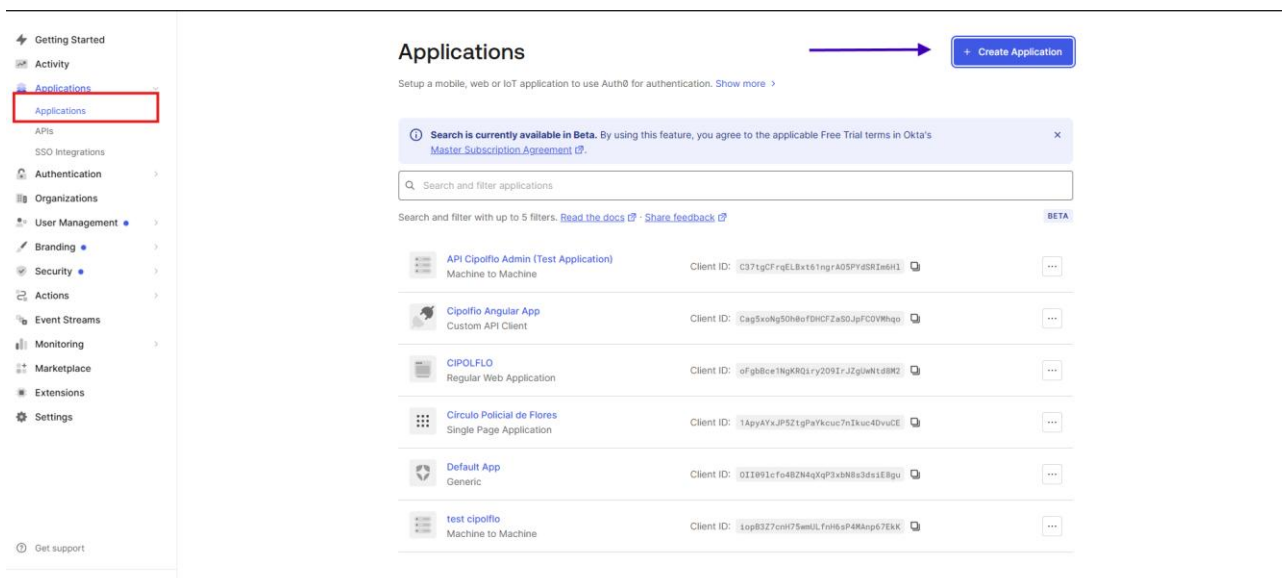


Imagen 1 - Panel de Auth0

Campo	Valor
Name	Circulo Policial de Flores
Tipo	Single Page Web Application



Create application



Select an application type or import from a Client ID Metadata URL to create an application. [Learn more](#)

[Create Manually](#)[Import from URL](#)

Name *

CIPOLFLO

You can change the application name later in the application settings.



This application is owned by a third party. Enable stricter security controls. [Learn more](#)

Choose an application type



Native

Mobile, desktop,
CLI and smart
device apps
running natively.

e.g.: iOS, Electron,
Apple TV apps

Single Page Web
Application

A JavaScript
front-end app that
uses an API.

e.g.: Angular,
React, Vue

Regular Web
Application

Traditional web
app using
redirects.

e.g.: Node.js
Express, ASP.NET,
Java, PHP

Machine to
Machine
Application

CLIs, daemons or
services running
on your backend.

e.g.: Shell script

Cancel

Create

Imagen 1 - Panel de Auth0

Configuración	Valor
Allowed Callback URLs	http://localhost:4200/ , https://staging.cipolflo.com.uy , https://cipolflo.com.uy
Allowed Logout URLs	http://localhost:4200/ , https://staging.cipolflo.com.uy , https://cipolflo.com.uy
Allowed Web Origins	http://localhost:4200/ , https://staging.cipolflo.com.uy , https://cipolflo.com.uy

2.3 Creación de Api

API, en Applications → APIs → Create API:

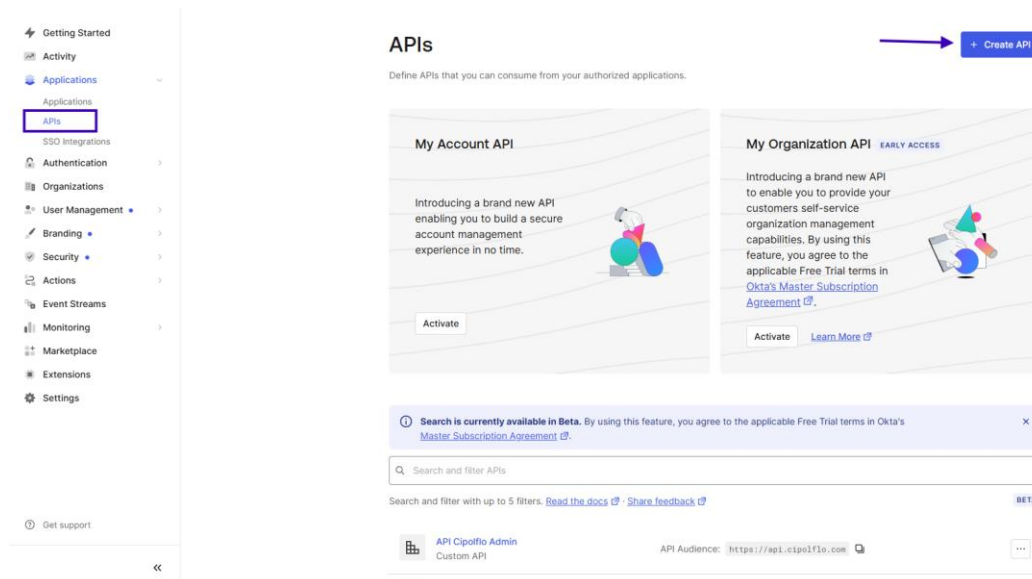


Imagen 2 - Panel de Auth0

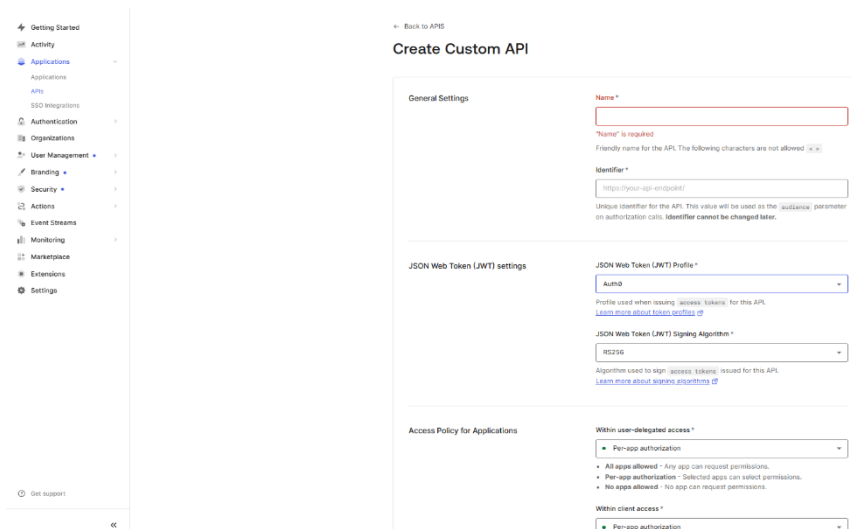


Imagen 3 - Panel de Auth0



Campo	Valor
Name	API Cipolfo Admin
Identifier	https://api.cipolflo.com
Signing Algorithm	RS256

RS256 usa criptografía asimétrica: Auth0 firma el token con una clave privada y Spring Boot valida la firma con la clave pública que Auth0 expone públicamente (vía JWKS), sin necesidad de compartir secretos entre ambos sistemas.

⚠ Importante

Domain, Client ID y el Identifier de la API son datos sensibles. No deben quedar hardcoded en el código ni subirse al repositorio — se configuran como variables de entorno (ver 2.3)



2.3 Variables de entorno

El Backend necesita de dos variables de entorno para validar los JWT emitidos por Auth0. El sistema las expone

```
auth0.issuer=${AUTH0_ISSUER_URI}
auth0.audience=${AUTH0_AUDIENCE}
```

Variable de entorno	Propiedad Spring	Función
AUTH0_ISSUER_URI	auth0.issuer	<i>Dominio del tenant de Auth0, usado para descubrir las claves públicas y validar el issuer del token</i>
AUTH0_AUDIENCE	auth0.audience	<i>Identificador de la API — el valor que el AudienceValidator verifica que esté presente en el claim del token</i>

Ambas claves deben estar definidas en el archivo .env (ver env.example) y como claves ocultas en el entorno ci para el ambiente de despliegue.

2.3 Bloque de comandos o valores técnicos

Formato recomendado para comandos, URLs, variables de entorno o valores exactos que el usuario debe copiar (útil para el manual de infraestructura):

```
auth0.issuer=${AUTH0_ISSUER_URI}
auth0.audience=${AUTH0_AUDIENCE}
```

2.4 Validación de JWT

SecurityConfig cumple tres responsabilidades: define cuales rutas requieren autenticación, configura CORS y arma el JwtDecoder con la validación de audiencia

Ver SecurityConfig en el repositorio cipolflo-server

Que hace cada parte del código:

Componente	Función
filterChain	Define las rutas públicas (/api/public/**, /api/health, /actuator/**) y exige autenticación para el resto (anyRequest().authenticated())
jwtDecoder	Construye el decoder a partir del issuerUri, agrega la validación estándar de issuer (createDefaultWithIssuer) y la combina con el AudienceValidator custom mediante DelegatingOAuth2TokenValidator



corsConfigurationSource	Permite los orígenes definidos en <code>cors.allowed-origins</code> , con métodos GET/POST/PUT/PATCH/DELETE/OPTIONS, credenciales habilitadas, y expone el header <code>Content-Disposition</code> (necesario para descargas de archivos, como los comprobantes PDF)
-------------------------	--

@EnableMethodSecurity está habilitado a nivel de clase, lo que permite en teoría usar anotaciones como @PreAuthorize en la sección de controllers, en este proyecto, no hay ninguna verificación de rol o de permiso ya que en esta situación solo existe un rol el cual es de administradores.

3. Preguntas frecuentes

¿Por qué Spring Boot rechaza un JWT con “invalid_token:The required audience is missing”?

El token fue emitido por Auth0, pero no incluye el Identifier de esta Api en su claim aud.

Tenemos que verificar que el frontend este solicitando el token correctamente con el audience correcto en los parámetros de autorización.

¿Por qué falla la validación, aunque el token parezca válido?

Tenemos que revisar que AUTH0_ISSUER_URI coincida exactamente con el dominio del tenant. Cualquier falta de letra o caracter en comparación al issuer del token de Auth0 hará que no funcione la validación.

4. Glosario

Término	Definición
Auth0	Plataforma de autenticación y autorización como servicio (IdP).
Tenant	Espacio de configuración del propio proyecto el cual esta dentro de Auth0.
Issuer	Entidad la cual emite el JWT, en este caso el dominio del tenant de Auth0.
Audience(aud)	Claim del JWT que identifica para que aplicación fue emitido el token.
RS256	Algoritmo usado para firmar y validar JWT
Resource Server	El rol que ocupa Spring boot para validar JWT y proteger endpoints del proyecto

5. Anexos

- Manual de alta de usuarios en Auth0